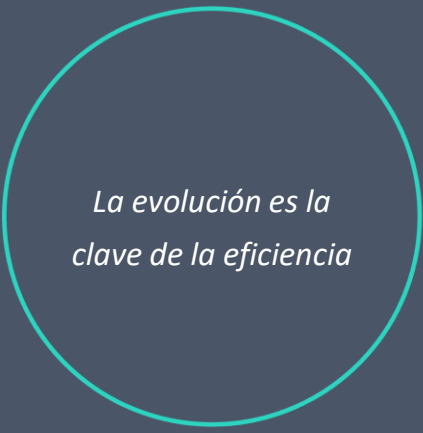


Fundamentos de Git y GitHub

Control de Versiones y Colaboración





*La evolución es la
clave de la eficiencia*

De la Historia a la Eficiencia

El control de versiones ha evolucionado significativamente para resolver los desafíos de la colaboración en desarrollo de software.

Sistemas Centralizados (CVCS):

- ► CVS y Subversion requerían conexión constante al servidor central

Sistemas Distribuidos (DVCS):

- ► Git permite trabajar sin conexión y con mayor flexibilidad
- ► Cada desarrollador tiene un repositorio completo
- ► Facilita el desarrollo paralelo y la colaboración global

Git vs. GitHub

Git: Software de control de versiones que se instala localmente. Es el motor que rastrea cambios y gestiona el historial del código.

GitHub: Plataforma web de alojamiento de repositorios Git. Proporciona herramientas de colaboración social y gestión de proyectos.

Funcionalidades de GitHub:

- ▶ Alojamiento de repositorios remotos
- ▶ Pull Requests y Code Reviews
- ▶ Issues y Project Management
- ▶ GitHub Actions para CI/CD





Ciclo de Vida del Código

El flujo de trabajo en Git se estructura en **tres niveles principales** más la interacción remota.



Working Directory: Archivos locales

`git add`



Staging Area: Cambios listos

`git commit`



Git Directory: Historial

`git push`



Remote: Repositorio central

`git pull`



Repositorios GitHub

Configuración y Tipos de Repositorios

Los repositorios en GitHub pueden configurarse de diferentes formas según las necesidades del proyecto y el equipo de desarrollo.

1

Repositorio Público

Visible para todos en internet. Ideal para proyectos de código abierto, portafolios y contribuciones a la comunidad.

2

Repositorio Privado

Solo accesible para colaboradores específicos. Perfecto para proyectos internos, código propietario y desarrollo confidencial.

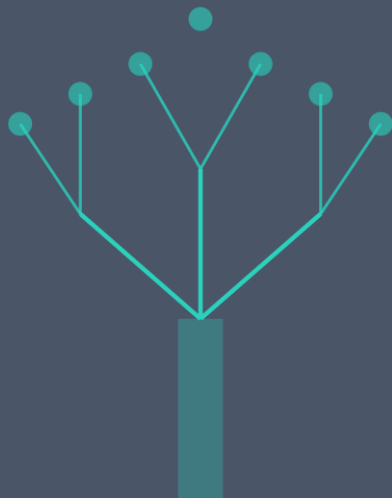
3

Fork

Copia personal de un repositorio existente. Permite experimentar y proponer cambios sin afectar el proyecto original.

Git Branching

Desarrollo Paralelo



Ramas: El Poder del Desarrollo Paralelo

Las ramas permiten trabajar en múltiples funcionalidades simultáneamente sin afectar el código principal.

01

Concepto de Rama

Un puntero móvil a un commit específico. Permite desarrollo aislado de nuevas funcionalidades.

02

Comandos Esenciales

`git branch`, `git checkout`, `git merge` para crear, cambiar y fusionar ramas.

03

Estrategia Git Flow

Modelo de branching con ramas específicas: develop, feature, release y hotfix para organizar el flujo.

04

Ventajas Clave

Desarrollo paralelo, aislamiento de cambios, facilita code reviews y

Dominando el Flujo

Comandos **esenciales** y **avanzados** para controlar el historial de tu código.



git add / commit

Preparar y registrar cambios locales



git push / pull

Sincronizar con repositorio remoto



git merge / rebase

Integrar cambios de diferentes ramas



git reset / revert

Deshacer cambios de forma segura

```
$git status
```

```
On branch main
```

```
Changes not staged for commit:
```

```
$git add .
```

```
$git commit -m "Add feature"
```

```
[main a1b2c3d] Add feature
```

```
2 files changed, 15 insertions
```

```
$git push origin main
```

```
Counting objects: 3, done.
```

```
Writing objects: 100%
```

```
To github.com:user/repo.git
```

```
x9y8z7w...a1b2c3d main -> main
```



Seguridad y Automatización

Buenas Prácticas

Seguridad y Automatización del Flujo de Trabajo

Implementar prácticas robustas para **proteger el código y automatizar procesos** en GitHub.

- 1 Autenticación SSH**
Conexión segura sin credenciales repetidas
- 2 Pull Requests y Code Reviews**
Validación de cambios antes de fusionar
- 3 GitHub Actions**
Automatización de CI/CD y pruebas
- 4 GitHub Projects**
Organización y seguimiento de tareas

Resumen y Próximos Pasos



Git es la base técnica, **GitHub** es la plataforma que potencia la colaboración global



La **práctica constante** de los comandos básicos es fundamental para dominar estas herramientas



Próximos pasos: Explorar GitHub Actions, contribuir a proyectos de código abierto, profundizar en estrategias avanzadas

*"El control de versiones no es solo una herramienta técnica, es la **base de la colaboración** en el desarrollo moderno."*

— Comunidad de Desarrolladores